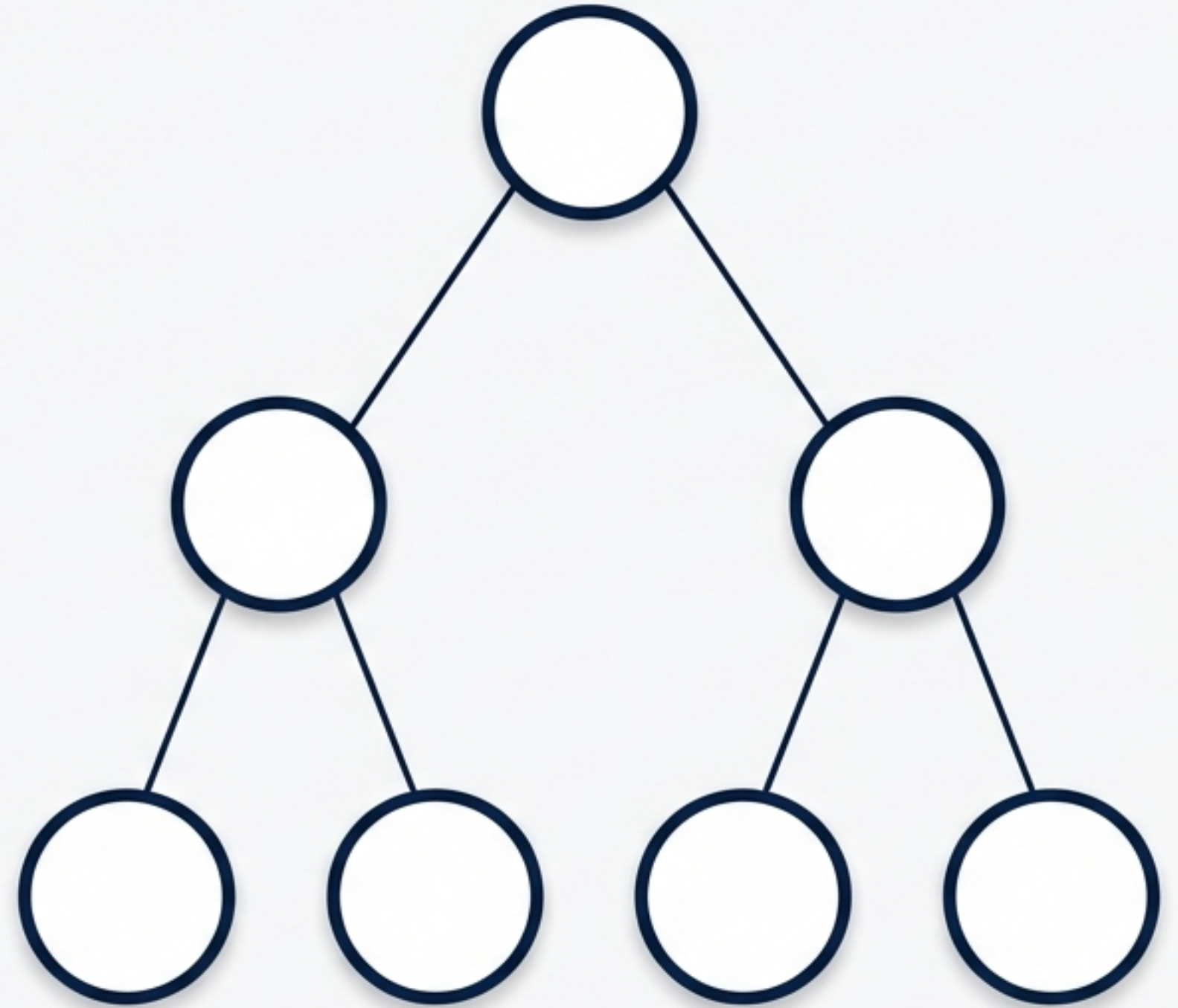


Mastering the Binary Search Tree (BST)

Concept, Insertion, and Search.

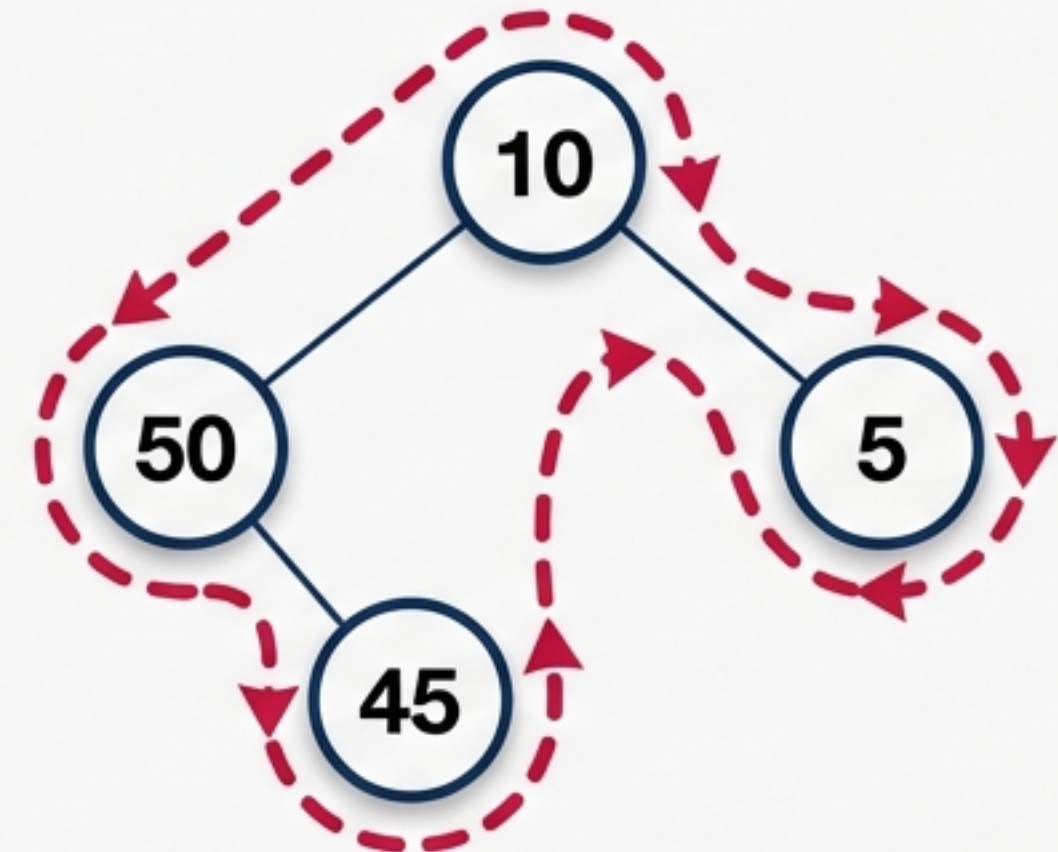
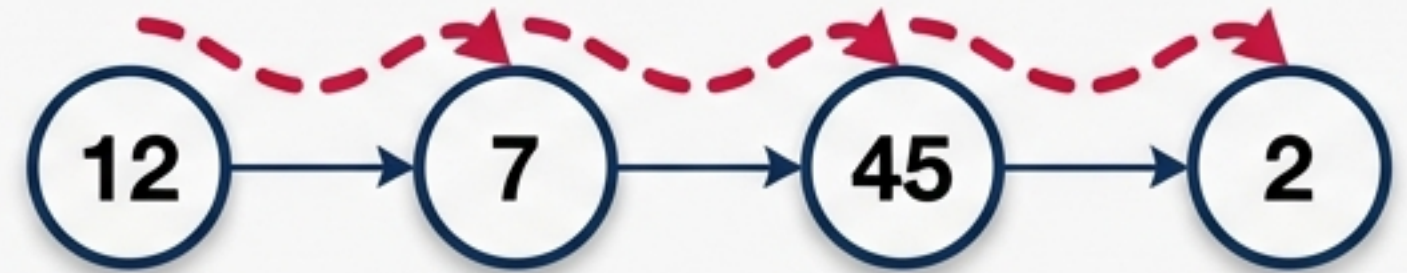
The 99% Rule: If an interview covers trees, there is a 99% chance the question relies on BST fundamentals.



The $O(N)$ Search Bottleneck

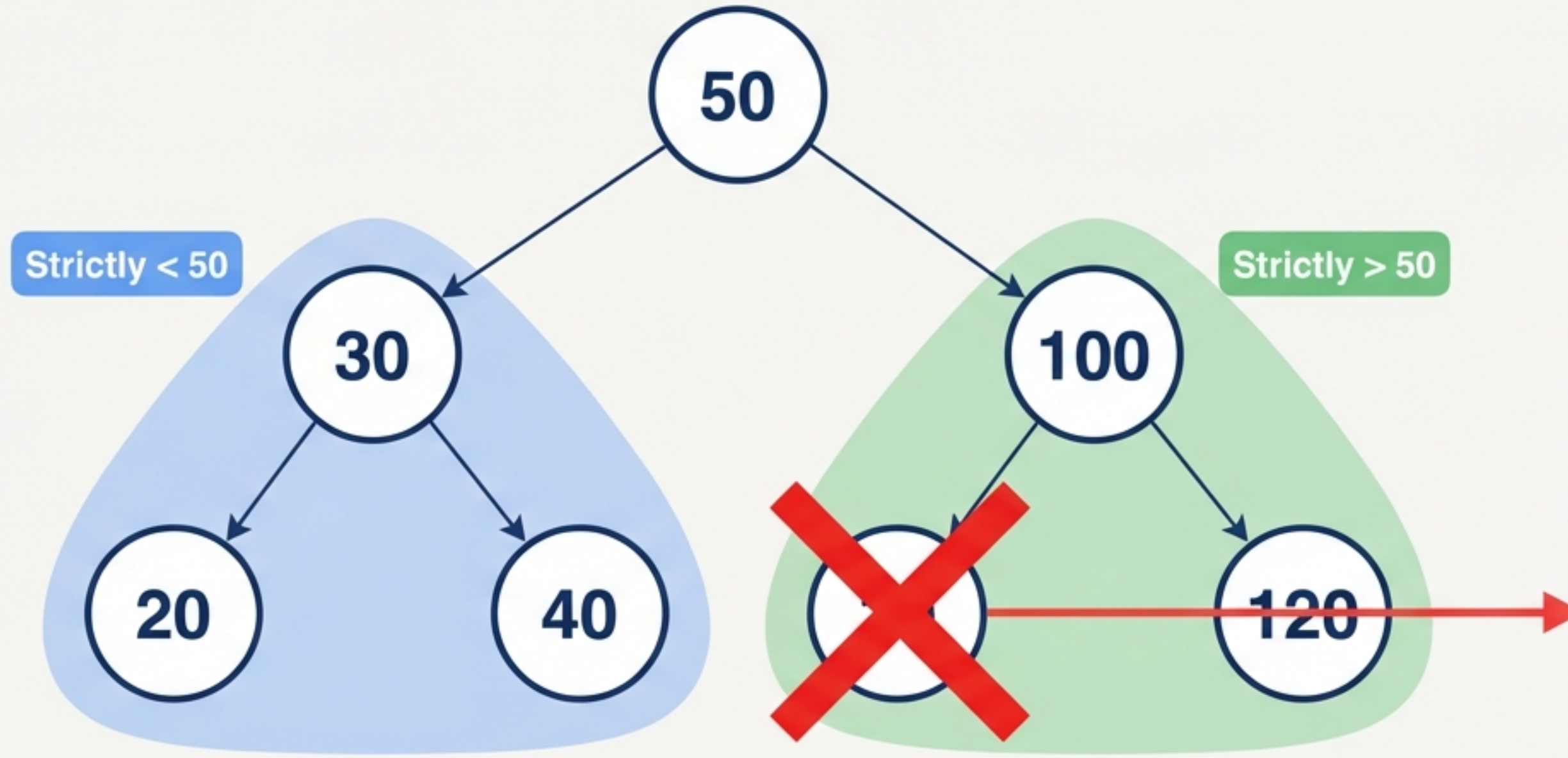
Standard structures like Arrays, Linked Lists, and basic Binary Trees lack sorting rules. Finding a specific value requires visiting every single node until a match is found.

If your dataset scales to millions, an $O(N)$ linear search becomes a critical performance failure.



The Golden Rule of the BST

Left Subtree < Root < Right Subtree



THE TRAP: The rule applies to the **ENTIRE subtree**, not just immediate children.

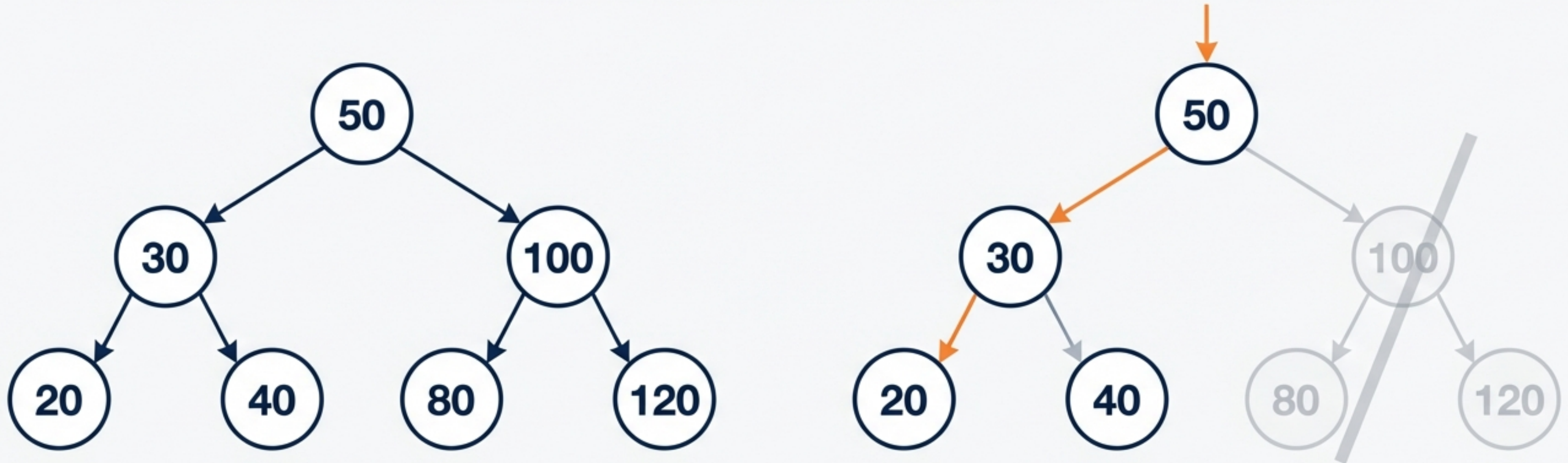
A left child of 100 must be smaller than 100, but still strictly greater than the main root (50).

Halving the Search Space

Every comparison in a BST eliminates half of the remaining tree.
You only make one comparison per level.

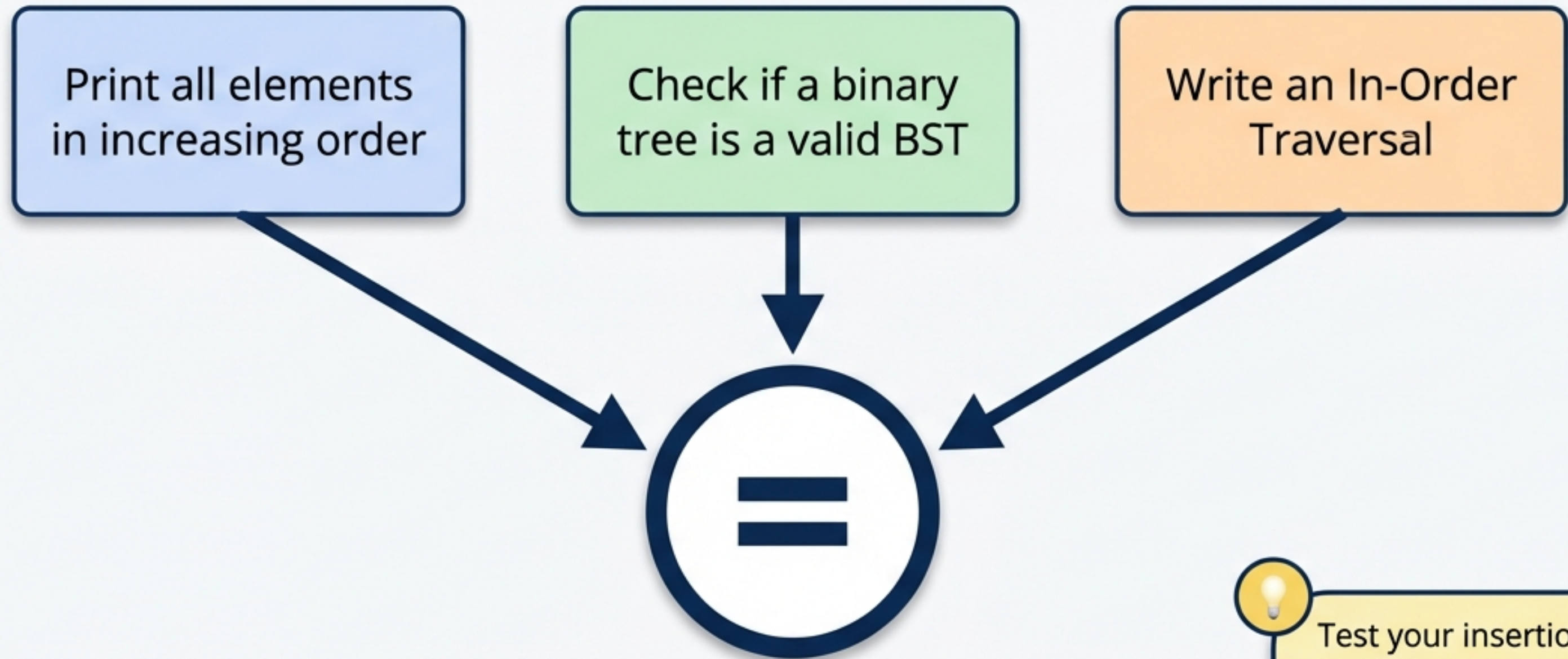
Max comparisons = Height of the Tree

Time Complexity drops dramatically from **$O(N)$** to **$O(\log N)$** .



Interview Secret: The "Same Question" Trick

Interviewers will disguise the prompt, but the underlying property never changes:
An In-Order Traversal of a valid BST will always print elements in strictly increasing order.



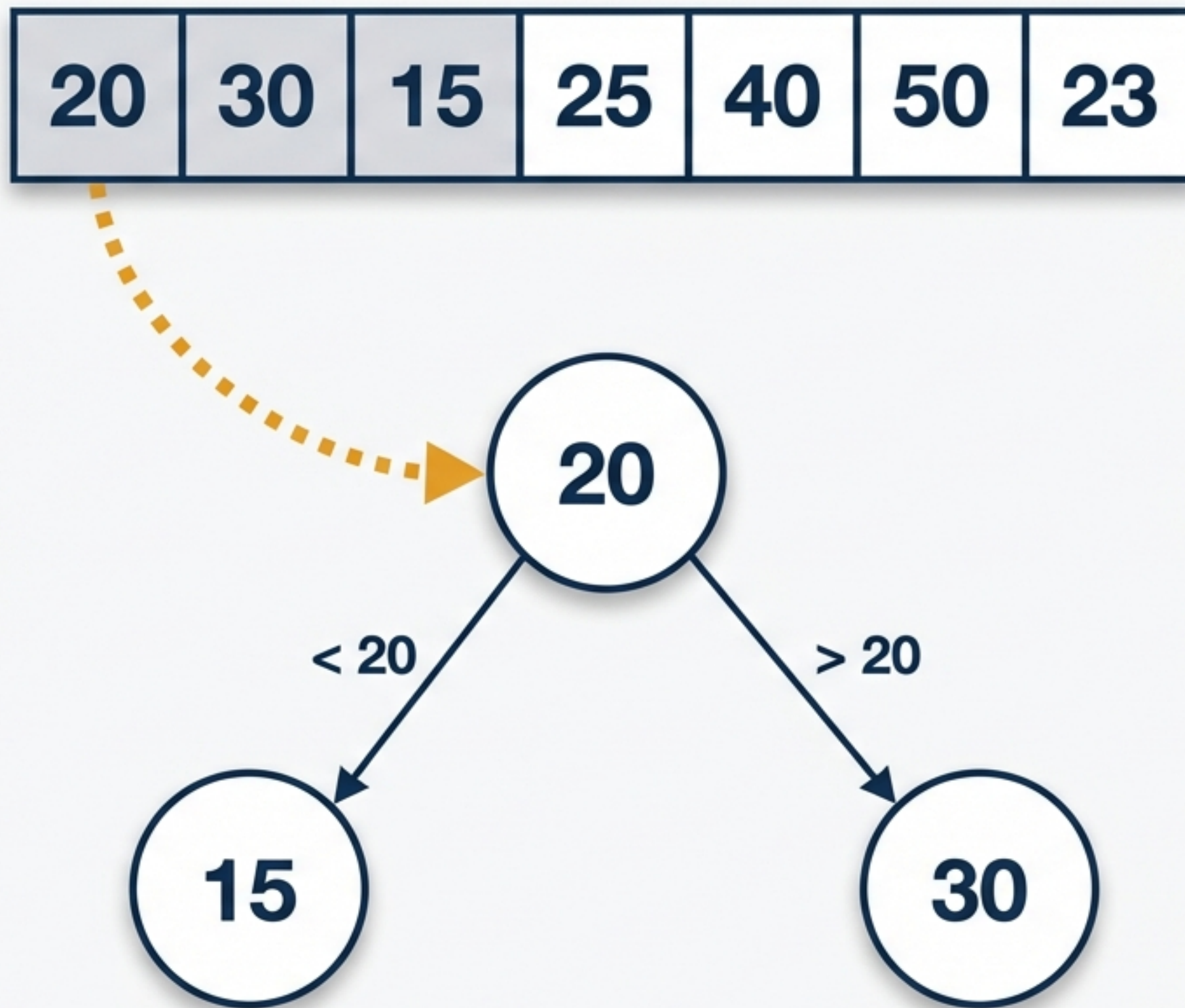
Exactly the same solution.



Test your insertion code by running an in-order traversal. If the output isn't sorted, your tree logic is broken.

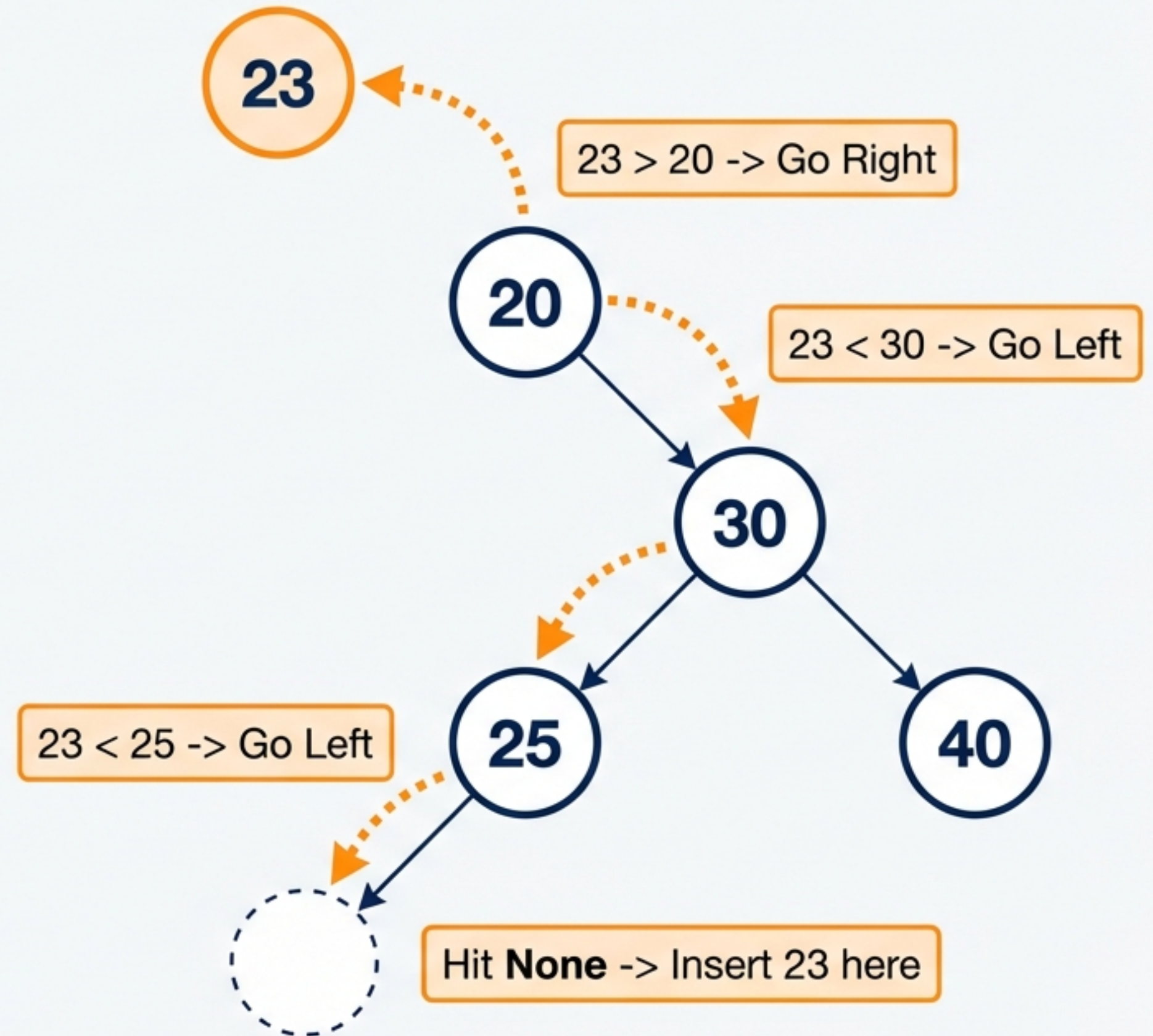
Visualizing Tree Creation (The Setup)

Creating a BST is simply repeated insertion. The first element becomes the root. Subsequent elements route **left** or **right** based on the **Golden Rule** until they hit empty space (None).



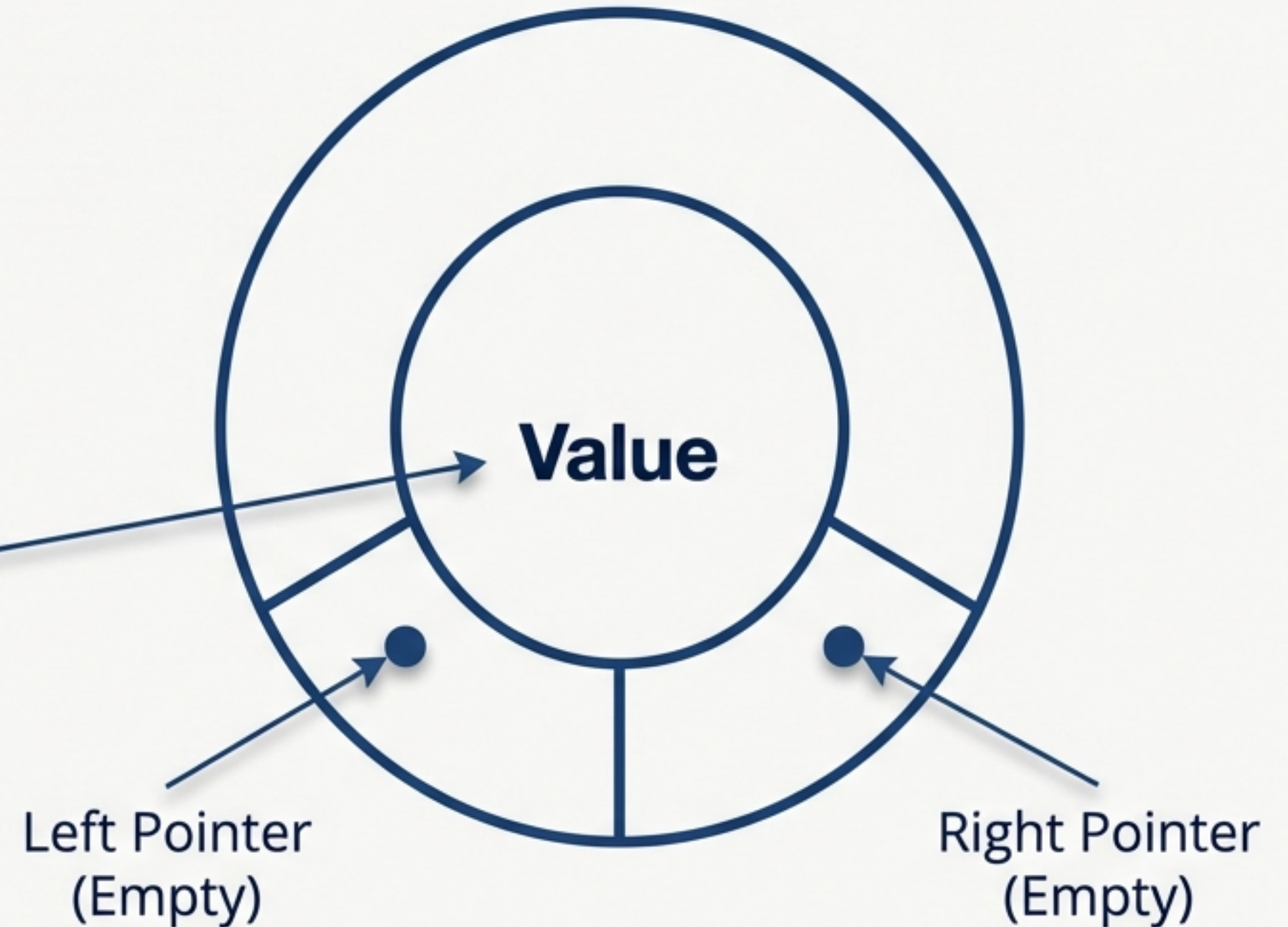
Tracing a Complex Insertion Path

We never shift existing nodes. We simply follow the left/right rules down the branches until we find a *None* pointer, then attach the new node.



Code Architecture: The Node Class

```
class Node:  
    def __init__(self, value):  
        self.left = None  
        self.right = None  
        self.data = value
```

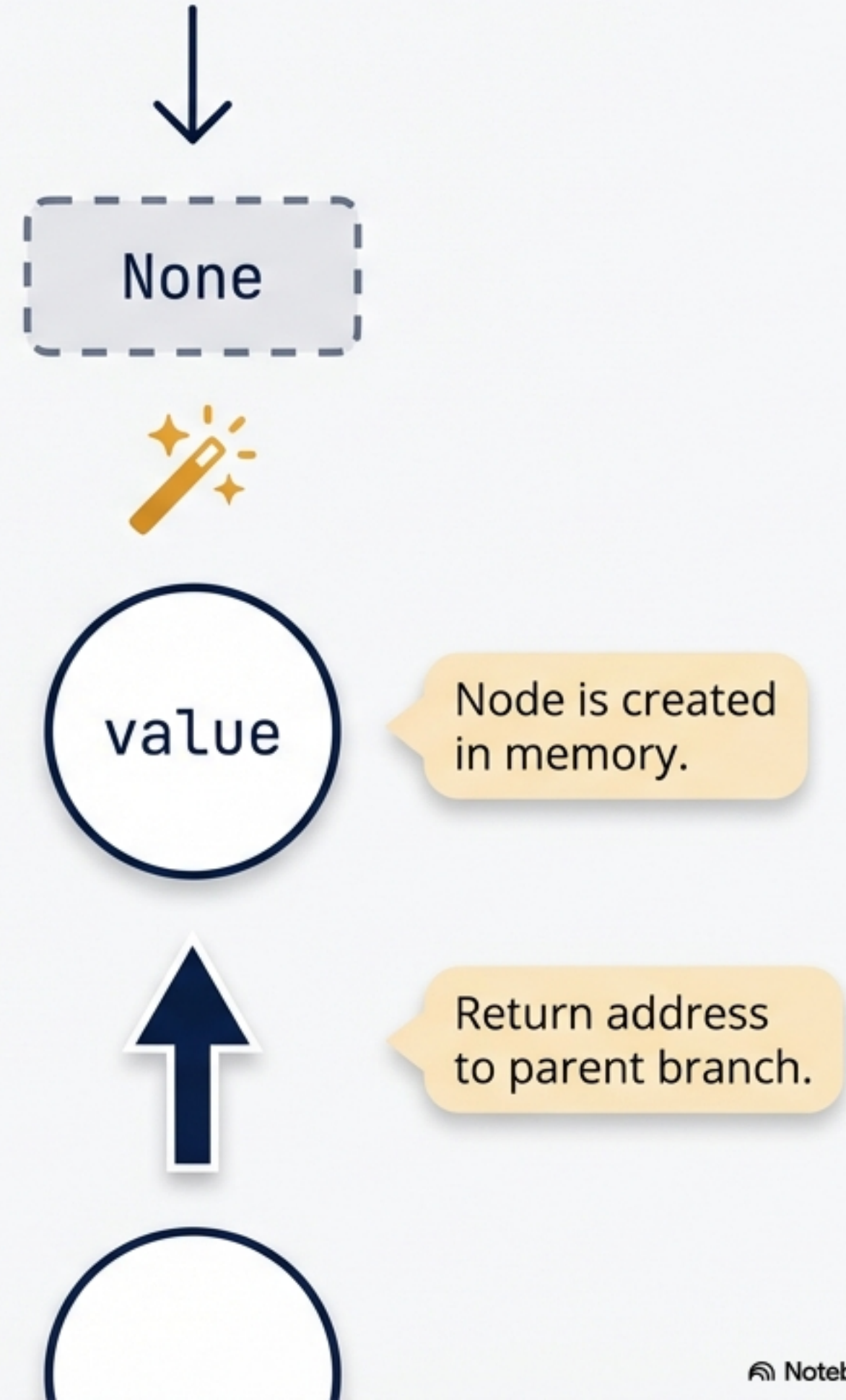


Every node starts as a leaf. Left and right pointers are initialized as None until we attach a child.

The Recursive Base Case

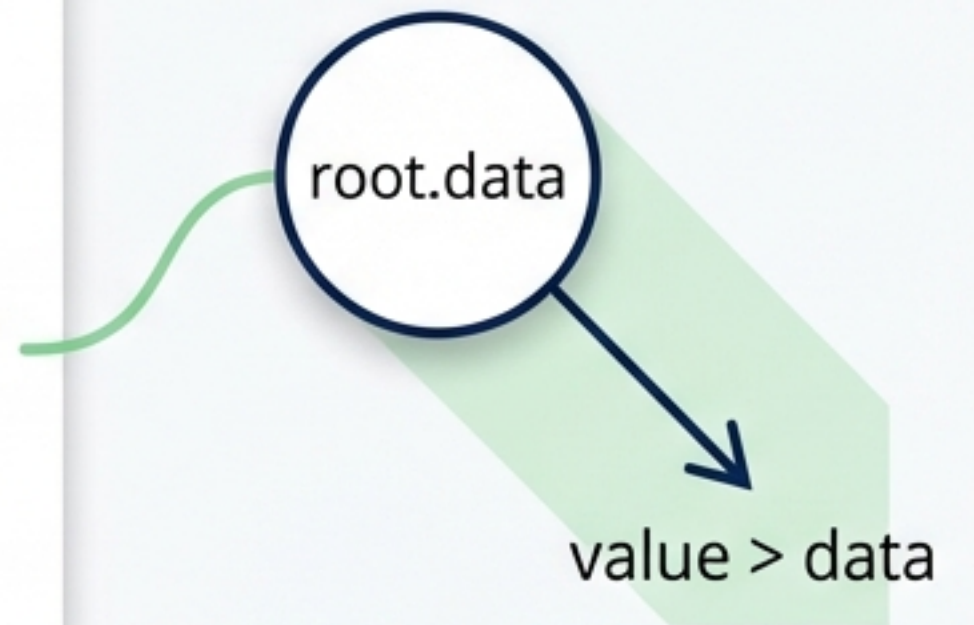
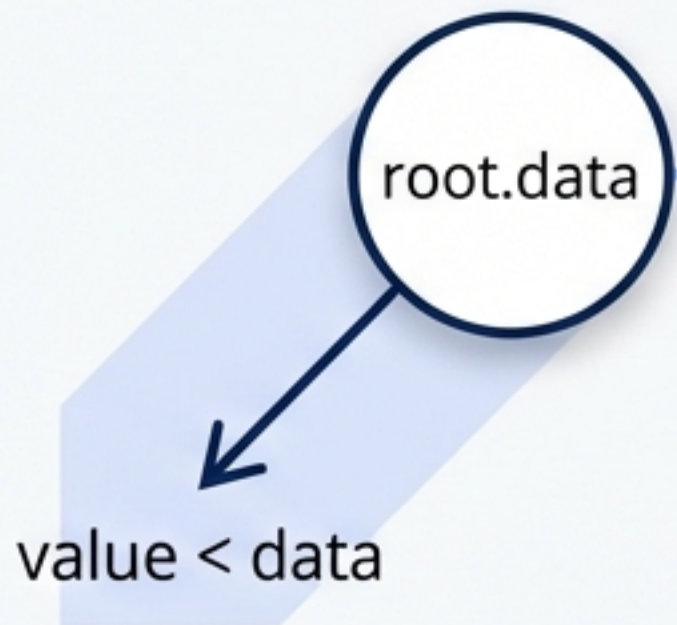
In recursion, write the termination step first. When our traversal hits a dead end (None), we allocate memory, create the node, and return it to the caller.

```
if root is None:  
    return Node(value)
```



The Routing Logic

```
if value < root.data:  
    root.left = insert(root.left, value)  
elif value > root.data:  
    root.right = insert(root.right, value)
```

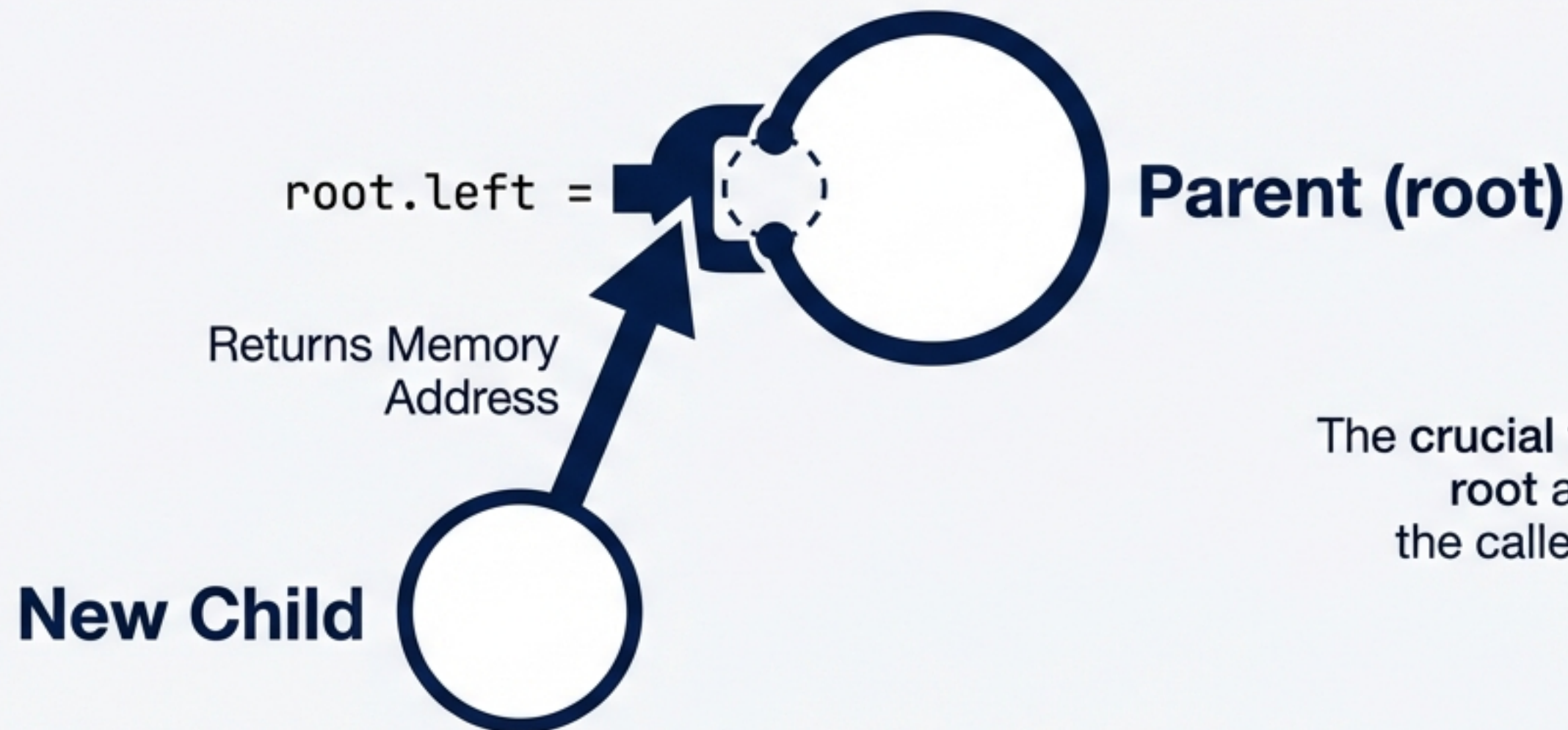


Note: If the value equals root.data, we do nothing. This implementation assumes unique elements and ignores duplicates.

The Reconnection Concept

Why do we assign the recursive call to **root.left**? The function creates a node and returns its address. The **parent node must actively update its pointer to 'catch' and connect this new node**.

```
root.left = insert(root.left, value)
```



The crucial final line: Always return the **root** at the end of the function so the caller receives the updated tree.

```
return root
```


The Search Algorithm (The Twin)

If you can write Insert, you can write Search. The traversal logic is identical. Instead of modifying the tree when hitting None, Search simply returns a success or failure state.

Insert Routing

```
if value < root.data:  
    root.left = insert(root.left, value)  
elif value > root.data:  
    root.right = insert(root.right, value)
```

100% Identical Logic

Search Routing

```
if value < root.data:  
    search(root.left, value)  
elif value > root.data:  
    search(root.right, value)
```


Terminations in Search

Explicitly return your control statements. If you fail to return after finding the element or hitting None, the code will continue to execute, attempt to check the .data of a None type, and crash the program.



```
if root is None:  
    return False # Dead end
```



```
if root.data == value:  
    return True # Found it
```



DANGER: NullPointerException. If you omit the 'return' keyword in the base cases, the program falls through to 'if value < root.data'. Checking '.data' on a NoneType object instantly crashes Python.

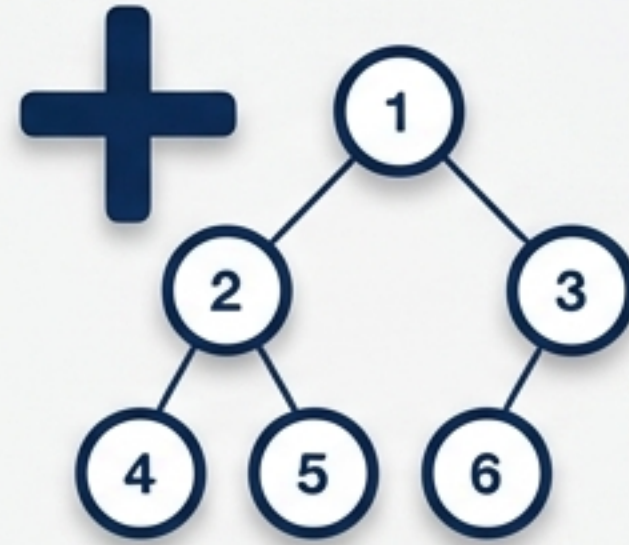
The Complexity Cheat Sheet



Single Search

$O(\log N)$

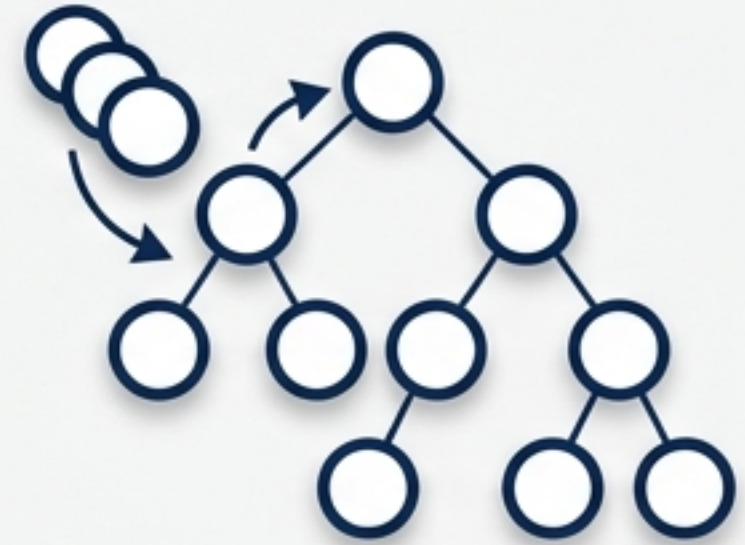
Halving the search space
at each step.



Single Insert

$O(\log N)$

Traversing the height of
the tree.



Complete Tree Creation

$O(N \log N)$

Inserting N elements into
the tree one by one.

The Next Challenge: Deletion

Inserting and Searching are elegant because they never shift existing nodes. Deletion breaks this rule.

Removing a node with 0, 1, or 2 children requires structural reorganization to maintain the Golden Rule.

Master insertion and search first. The complexity of deletion builds entirely on these foundations.

